Sarcasm Detection Model

Final Project Report

[**Alex Oteng, Justin Sheller**]

December 18, 2025

Contents

1 Introduction

1.1 Problem Description

Sarcasm detection is a challenging natural language processing (NLP) task that requires understanding the contradiction between literal meaning and intended meaning. Sarcasm often relies on implicit cues such as punctuation, word choice, and phrasing rather than explicit sentiment indicators.

For example:

- Literal: "I love this weather." (It is sunny outside.)
- Sarcastic: "I love this weather." (There is a hurricane)

Since Transformer-based architectures (e.g., BERT, GPT, RoBERTa) are not permitted for this assignment, our approach relies on lexical and structural cues to detect sarcasm.

1.2 Approach Overview

Our solution combines:

1. Custom preprocessing that preserves punctuation
2. TF-IDF vectorization with unigrams and bigrams
3. A soft-voting ensemble classifier
4. A pipeline-based architecture for training and inference

2 Data Exploration and Preprocessing

2.1 Dataset Statistics

- **Training Set**: 21,464 samples
- **Validation Set**: 716 samples
- **Test Set**: 966 samples

Each sample consists of a short text phrase and a binary label (0 = non-sarcastic, 1 = sarcastic). The dataset is relatively balanced.

2.2 Custom Preprocessing

2.2.1 Preprocessing Code

```
text = text.lower()
text = re.sub(f"([{string.punctuation}])", r" \1 ", text)
text = re.sub(r"\s+", " ", text).strip()
```

2.2.2 Motivation

Standard preprocessing pipelines often remove punctuation entirely. However, punctuation is a strong signal for sarcasm:

- "Great." (neutral)
- "Great!!!" (exaggeration or sarcasm)
- "Great..." (disappointment or irony)

Instead of removing punctuation marks, we isolate them as individual tokens.

2.2.3 Example

- Input: `Great!!!`
- Output: `great ! ! !`

This allows the model to learn punctuation patterns associated with sarcasm.

2.3 Data Cleaning

- Remove missing text or labels
- Convert labels to integers
- Apply lowercase normalization

3 Feature Engineering

3.1 TF-IDF Vectorization

```
TfidfVectorizer(
    ngram_range=(1, 2),
    min_df=2,
    preprocessor=custom_preprocessor
)
```

TF-IDF transforms text into numerical features based on term importance. Words that appear frequently in a document but rarely across the corpus receive higher scores.

3.2 N-Grams

- **Unigrams**: single words (e.g., "yeah")
- **Bigrams**: word pairs (e.g., "yeah right")

Bigrams are particularly important for sarcasm, which often resides in short phrases rather than individual words.

3.3 Feature Selection

Using `min_df=2` filters rare tokens, reducing noise while preserving meaningful features.

4 Model Architecture and Selection

4.1 Models Considered

1. Logistic Regression
2. Random Forest
3. Gradient Boosting
4. Ensemble Voting Classifier

4.2 Final Ensemble Architecture

Our final model uses soft voting to average class probabilities from three classifiers.

4.2.1 Logistic Regression

- Linear classifier with L2 regularization
- Fast and interpretable
- Struggles with non-linear patterns

4.2.2 Random Forest

- Ensemble of decision trees
- Captures non-linear feature interactions
- Risk of overfitting without depth control

4.2.3 Gradient Boosting

- Sequential tree-based learning
- Strong performance on difficult cases
- Requires careful tuning

4.3 Soft Voting

Instead of majority voting, soft voting averages predicted probabilities:

$$P(y = 1) = \frac{1}{3}(P_{LR} + P_{RF} + P_{GB})$$

This method captures model confidence and improves accuracy.

5 Pipeline Architecture

```
pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(...)),
    ('classifier', ensemble)
])
```

Pipelines ensure consistent preprocessing during training and inference and simplify deployment.

6 Training Methodology

6.1 Training Procedure

1. Load training, validation, and test sets
2. Fit pipeline on training data
3. Save trained model using `joblib`

6.2 Evaluation Metrics

- Accuracy
- Precision, Recall, F1-score
- Macro-averaged F1

6.3 Regularization

- Logistic Regression: L2 regularization
- Random Forest: Maximum depth constraint
- Gradient Boosting: Learning rate and tree depth
- TF-IDF: Minimum document frequency

7 Experiments and Results

7.1 Final Performance

Dataset	Accuracy	F1-score
Validation	0.8282	0.8289
Test	0.8209	0.8080

Table 1: Model Performance

7.2 Ablation Studies

- Removing punctuation isolation reduced performance
- Bigrams improved F1-score by approximately 6%
- Soft voting outperformed hard voting

7.3 Error Analysis

The model shows higher precision for non-sarcastic samples, indicating conservative sarcasm predictions.

8 Discussion

8.1 Strengths

- Effective punctuation modeling
- Robust ensemble architecture
- Strong generalization performance

8.2 Limitations

- No contextual awareness
- English-only training data
- Short-text optimization

8.3 Future Improvements

- Word embeddings (Word2Vec, GloVe)
- Character-level features
- Cross-validation
- Class weighting

9 Conclusion

This project demonstrates that strong sarcasm detection performance can be achieved without transformer models by leveraging careful preprocessing, feature engineering, and ensemble learning.

10 References

- scikit-learn
- pandas
- numpy
- joblib
- scipy